

POAI - LAB RECORD

8- QUEENS PROBLEM

```
def isSafe(mat, r, c):
    # Check column
    for i in range(r):
        if mat[i][c] == '&#39;Q&#39;:
            return False

    # Check upper left diagonal
    i, j = r, c
    while i &gt;= 0 and j &gt;= 0:
        if mat[i][j] == '&#39;Q&#39;:
            return False
        i -= 1
        j -= 1

    # Check upper right diagonal
    i, j = r, c
    while i &gt;= 0 and j &lt; len(mat):
        if mat[i][j] == '&#39;Q&#39;:
            return False
        i -= 1
        j += 1

    return True

def printSolution(mat):
    for row in mat:
        print(str(row).replace('&#39;,&#39;, '&#39;&#39;').replace('&quot;&#39;&quot;',
            '&quot;&quot;'))
        print()

def nQueen(mat, r):
    if r == len(mat):
        printSolution(mat)
```

```
return
```

```
for i in range(len(mat)):
    if isSafe(mat, r, i):
        mat[r][i] = 'Q'
        nQueen(mat, r + 1)
        mat[r][i] = '-' # backtrack

if __name__ == '__main__':
    N = int(input('Enter number of queens: '))
    mat = [['-' for _ in range(N)] for _ in range(N)]
    nQueen(mat, 0)
```

Output:

For input:

None

Enter number of queens: 4

Output:

None

- Q - -

- - - Q

Q - - -

- - Q -

- - Q -

Q - - -

- - - Q

- Q - -

DEPTH FIRST SEARCH

```
import networkx as nx

# FUNCTION TO SOLVE DFS
def solveDFS(graph, v, visited):
    visited.add(v)
    print(v, end='; ')

    for neighbour in graph[v]:
        if neighbour not in visited:
            solveDFS(graph, neighbour, visited)

# Create a graph using NetworkX
g = nx.DiGraph()

# Add edges for the graph
g.add_edges_from([('A', 'B'), ('A', 'C'), ('C', 'G'), ('B', 'D'), ('B', 'E'), ('D', 'F'), ('A', 'E')])

# Visualize the graph
nx.draw(g, with_labels=True)

# Solve DFS for the graph starting from vertex A
print('Following is DFS from (starting from vertex A):')
visited = set()
solveDFS(g, 'A', visited)
```

Output:

Following is DFS starting from vertex A:
A B D F E C G

DEPTH FIRST SEARCH – WATER JUG PROBLEM

```
from collections import deque
```

```
def DFS(a, b, target):
```

```
    m = {}
```

```
    isSolvable = False
```

```
    path = []
```

```
    q = deque()
```

```
    q.append((0, 0))
```

```
    while (len(q) > 0):
```

```
        u = q.popleft()
```

```
        if ((u[0], u[1]) in m):
```

```
            continue
```

```
        if ((u[0] > a or u[1] > b or u[0] < 0 or u[1] < 0)):
```

```
            continue
```

```
        path.append([u[0], u[1]])
```

```
        m[(u[0], u[1])] = 1
```

```
        if (u[0] == target or u[1] == target):
```

```
            isSolvable = True
```

```
        if (u[0] == target):
```

```
            if (u[1] != 0):
```

```
                path.append([u[0], 0])
```

```
            else:
```

```
                if (u[0] != 0):
```

```
                    path.append([0, u[1]])
```

```
        sz = len(path)
```

```
        for i in range(sz):
```

```
            print("<quot;<quot;, path[i][0], <quot;<quot;, path[i][1], <quot;<quot;)
```

```
            break
```

```
    # add all next states
```

```
q.append([u[0], b])
q.append([a, u[1]])
```

```
for ap in range(max(a, b) + 1):
    c = u[0] + ap
    d = u[1] - ap
```

```
if (c == a or (d == 0 and d >= 0)):
    q.append([c, d])
```

```
c = u[0] - ap
d = u[1] + ap
```

```
if ((c == 0 and c >= 0) or d == b):
    q.append([c, d])
```

```
q.append([a, 0])
q.append([0, b])
```

```
if (not isSolvable):
    print("No solution")
```

```
Jug1, Jug2, target = 4, 3, 2
print("Path from initial state to solution state ::")
DFS(Jug1, Jug2, target)
```

Output:

Path from initial state to solution state:

```
(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
(2, 0)
```

MINIMAX ALGORITHM

```
from math import inf as infinity
from random import choice
import platform
import time
from os import system
```

```
HUMAN = -1
COMP = +1
```

```
board = [
[0, 0, 0],
[0, 0, 0],
[0, 0, 0]
]
```

```
def evaluate(state):
if wins(state, COMP):
```

```
score = +1
elif wins(state, HUMAN):
score = -1
else:
score = 0
return score
```

```
def wins(state, player):
win_state = [
[state[0][0], state[0][1], state[0][2]],
[state[1][0], state[1][1], state[1][2]],
[state[2][0], state[2][1], state[2][2]],
[state[0][0], state[1][0], state[2][0]],
[state[0][1], state[1][1], state[2][1]],
[state[0][2], state[1][2], state[2][2]],
[state[0][0], state[1][1], state[2][2]],
[state[2][0], state[1][1], state[0][2]],
]
if [player, player, player] in win_state:
return True
```

```

else:
    return False

def game_over(state):
    return wins(state, HUMAN) or wins(state, COMP)

def empty_cells(state):
    cells = []
    for x, row in enumerate(state):
        for y, cell in enumerate(row):
            if cell == 0:

cells.append([x, y])
    return cells

def valid_move(x, y):
    if [x, y] in empty_cells(board):
        return True
    else:
        return False

def set_move(x, y, player):
    if valid_move(x, y):
        board[x][y] = player
        return True
    else:
        return False

def minimax(state, depth, player):
    if player == COMP:
        best = [-1, -1, -infinity]
    else:
        best = [-1, -1, +infinity]

    if depth == 0 or game_over(state):
        score = evaluate(state)
        return [-1, -1, score]

    for cell in empty_cells(state):
        x, y = cell[0], cell[1]

```

```

state[x][y] = player
score = minimax(state, depth - 1, -player)
state[x][y] = 0

score[0], score[1] = x, y

if player == COMP:
    if score[2] > best[2]:
        best = score
    else:
        if score[2] < best[2]:
            best = score

return best

def clean():
    os_name = platform.system().lower()
    if 'windows' in os_name:
        system('cls')
    else:
        system('clear')

def render(state, c_choice, h_choice):
    chars = {
        -1: h_choice,
        +1: c_choice,
        0: ' ',
    }
    str_line = ''

    print('\n' + str_line)
    for row in state:
        for cell in row:
            symbol = chars[cell]
            print(f' | {symbol} |', end='')

    print('\n' + str_line)

def ai_turn(c_choice, h_choice):
    depth = len(empty_cells(board))

```



```
if depth == 0 or game_over(board):  
    return
```

```
clean()  
print(f'Computer turn [{c_choice}]')  
render(board, c_choice, h_choice)
```

```
if depth == 9:  
    x = choice([0, 1, 2])  
    y = choice([0, 1, 2])  
else:  
    move = minimax(board, depth, COMP)  
    x, y = move[0], move[1]
```

```
set_move(x, y, COMP)  
time.sleep(1)
```

```
def human_turn(c_choice, h_choice):  
    depth = len(empty_cells(board))  
    if depth == 0 or game_over(board):  
        return
```

```
    move = -1  
    moves = {  
        1: [0, 0], 2: [0, 1], 3: [0, 2],  
        4: [1, 0], 5: [1, 1], 6: [1, 2],  
        7: [2, 0], 8: [2, 1], 9: [2, 2],  
    }
```

```
clean()  
print(f'Human turn [{h_choice}]')  
render(board, c_choice, h_choice)
```

```
while move < 1 or move > 9:  
    try:  
        move = int(input('Use numpad (1..9): '))  
        coord = moves[move]  
        can_move = set_move(coord[0], coord[1], HUMAN)
```

```

if not can_move:
    print('Bad move')
    move = -1
except (EOFError, KeyboardInterrupt):
    print('Bye')
    exit()
except (KeyError, ValueError):
    print('Bad choice')

def main():
    clean()
    h_choice = 'X or O'
    c_choice = 'X or O'
    first = 'if human is the first'

    while h_choice != 'O' and h_choice != 'X':
        try:
            print('')
            h_choice = input('Choose X or O\nChosen: ').upper()

        except (EOFError, KeyboardInterrupt):
            print('Bye')
            exit()
        except (KeyError, ValueError):
            print('Bad choice')

    if h_choice == 'X':
        c_choice = 'O'
    else:
        c_choice = 'X'

    while first != 'Y' and first != 'N':
        try:
            first = input('First to start? [y/n]: ').upper()
        except (EOFError, KeyboardInterrupt):
            print('Bye')
            exit()
        except (KeyError, ValueError):
            print('Bad choice')

```

```

while len(empty_cells(board)) > 0 and not game_over(board):
    if first == 'N':
        ai_turn(c_choice, h_choice)
        first = 'O'
    human_turn(c_choice, h_choice)
    ai_turn(c_choice, h_choice)

    if wins(board, HUMAN):
        clean()
        print(f'Human turn [{h_choice}]')
        render(board, c_choice, h_choice)

    print('YOU WIN!')
    elif wins(board, COMP):
        clean()
        print(f'Computer turn [{c_choice}]')
        render(board, c_choice, h_choice)
        print('YOU LOSE!')
    else:
        clean()
        render(board, c_choice, h_choice)
        print('DRAW!')

if __name__ == '__main__':
    main()

```

Output:

Choose X or O: X

First to start? [Y/N]: Y

Human turn [X]

```

-----
|  |  |  |
-----
|  |  |  |
-----
|  |  |  |
-----

```

Use numpad (1..9): 5

Computer turn [O]

```
-----  
|  ||  ||  |  
-----  
|  ||X||  |  
-----  
|  ||  ||O|  
-----
```